

```
In [1]: # =====
# 1. Import Required Libraries
# =====

import tensorflow as tf                # Main deep learning
from tensorflow.keras import datasets, layers, models # Modules for
import matplotlib.pyplot as plt        # Library for visualiza

# =====
# 2. Load the CIFAR-10 Dataset
# =====

# CIFAR-10 contains 60,000 images of size 32x32 with 10 classes
# 50,000 images are used for training
# 10,000 images are used for testing

(train_images, train_labels), (test_images, test_labels) = datasets

# =====
# 3. Normalize the Image Data
# =====

# Pixel values in images range from 0 to 255
# Neural networks train better when values are between 0 and 1
# So we divide by 255

train_images = train_images / 255.0
test_images = test_images / 255.0

# =====
# 4. Define Class Names
# =====

# CIFAR-10 has 10 classes

class_names = [
    'airplane',
    'automobile',
    'bird',
    'cat',
    'deer',
    'dog',
    'frog',
    'horse',
    'ship',
    'truck'
]

# =====
# 5. Display Some Training Images
```

```

# =====

# This helps verify that the dataset loaded correctly

plt.figure(figsize=(10,10))

for i in range(9):

    plt.subplot(3,3,i+1)          # Create a 3x3 grid
    plt.imshow(train_images[i])   # Show the image
    plt.title(class_names[train_labels[i][0]]) # Display image label
    plt.axis('off')               # Hide axis

plt.show()

# =====
# 6. Build the CNN Model
# =====

# Sequential model means layers are stacked one after another

model = models.Sequential()

# ---- First Convolution Layer ----
# 32 filters of size 3x3
# ReLU activation introduces non-linearity

model.add(layers.Conv2D(
    32,                # number of filters
    (3,3),             # filter size
    activation='relu', # activation function
    input_shape=(32,32,3) # image size (height, width, channels)
))

# ---- First Max Pooling Layer ----
# Reduces spatial size to reduce computation

model.add(layers.MaxPooling2D((2,2)))

# ---- Second Convolution Layer ----
# Increase number of filters to learn more complex features

model.add(layers.Conv2D(
    64,
    (3,3),
    activation='relu'
))

# ---- Second Max Pooling Layer ----

model.add(layers.MaxPooling2D((2,2)))

```

```

# ---- Third Convolution Layer ----

model.add(layers.Conv2D(
    64,
    (3,3),
    activation='relu'
))

# =====
# 7. Flatten Layer
# =====

# Converts 3D feature maps into a 1D vector
# Required before feeding data into Dense layers

model.add(layers.Flatten())

# =====
# 8. Fully Connected (Dense) Layers
# =====

# Hidden dense layer

model.add(layers.Dense(
    64,
    activation='relu'
))

# Output layer
# 10 neurons because CIFAR-10 has 10 classes

model.add(layers.Dense(10))

# =====
# 9. Compile the Model
# =====

model.compile(

    optimizer='adam', # optimization algorithm

    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    # loss function for multi-class classification

    metrics=['accuracy'] # evaluation metric
)

# =====
# 10. Display Model Architecture

```

```

# =====
# Shows layer types, output shapes, and parameters

model.summary()

# =====
# 11. Train the Model
# =====

# The model learns patterns from the training dataset

history = model.fit(

    train_images,      # training images
    train_labels,      # corresponding labels

    epochs=10,         # number of training iterations

    validation_data=(test_images, test_labels) # validation dataset
)

# =====
# 12. Evaluate Model Performance
# =====

# Evaluate model using test dataset

test_loss, test_acc = model.evaluate(
    test_images,
    test_labels
)

print("Test Accuracy:", test_acc)

# =====
# 13. Make Predictions
# =====

# Predict class probabilities for test images

predictions = model.predict(test_images)

# Example prediction for first test image

import numpy as np

predicted_label = np.argmax(predictions[0])

print("Predicted Class:", class_names[predicted_label])
print("Actual Class:", class_names[test_labels[0][0]])

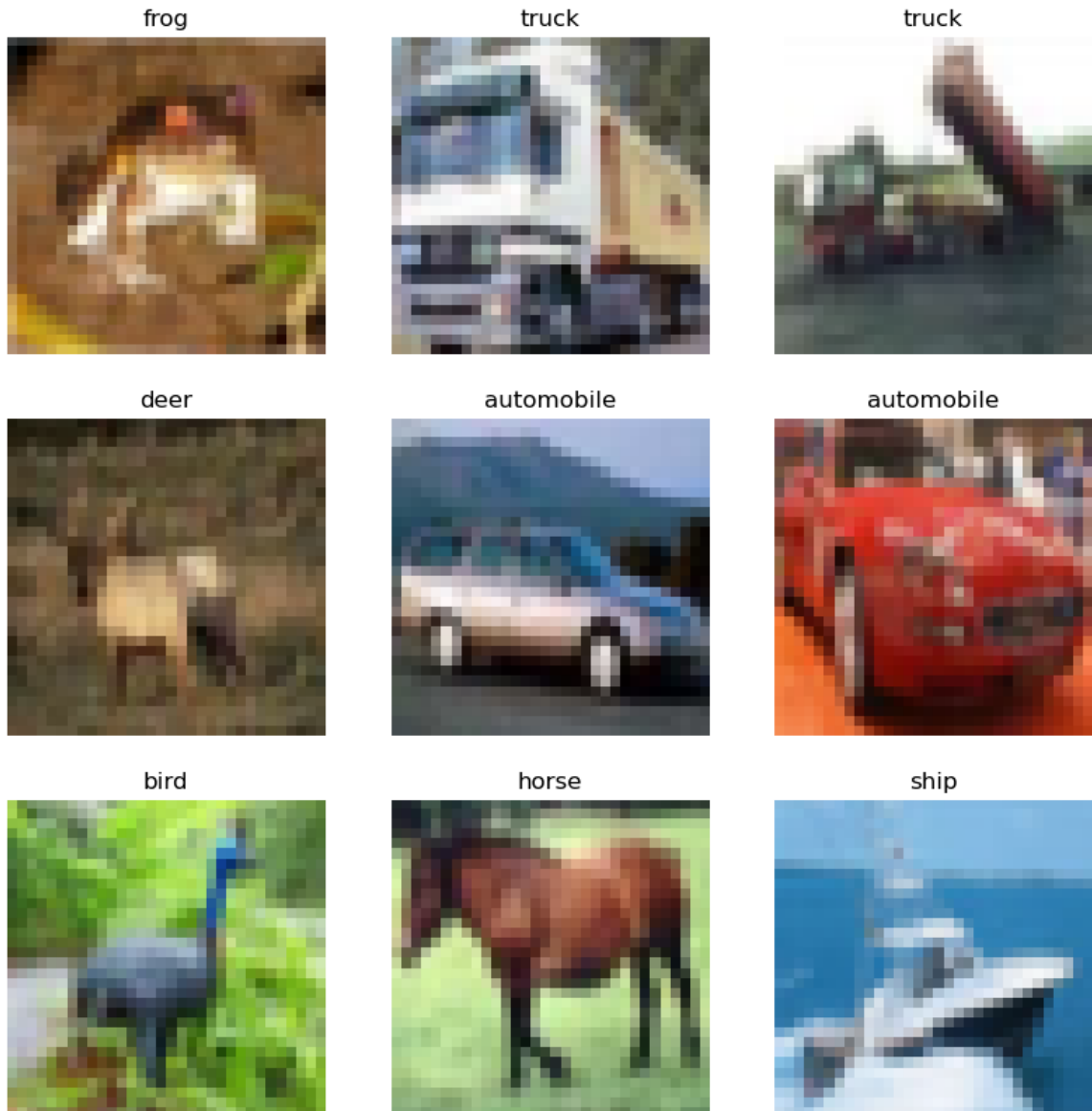
```

```
# =====
# 14. Display the Predicted Image
# =====

plt.imshow(test_images[0])
plt.title("Predicted: " + class_names[predicted_label])
plt.axis('off')
plt.show()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>

170498071/170498071 ————— 407s 2us/step



C:\Users\user\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape
conv2d (Conv2D)	(None, 30, 30, 32)
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)
conv2d_1 (Conv2D)	(None, 13, 13, 64)
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)
conv2d_2 (Conv2D)	(None, 4, 4, 64)
flatten (Flatten)	(None, 1024)
dense (Dense)	(None, 64)
dense_1 (Dense)	(None, 10)

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
1563/1563  **23s** 13ms/step - accuracy: 0.4596 - loss: 1.4836 - val_accuracy: 0.5131 - val_loss: 1.3438

Epoch 2/10
1563/1563  **19s** 12ms/step - accuracy: 0.6021 - loss: 1.1231 - val_accuracy: 0.6400 - val_loss: 1.0253

Epoch 3/10
1563/1563  **19s** 12ms/step - accuracy: 0.6570 - loss: 0.9739 - val_accuracy: 0.6601 - val_loss: 0.9620

Epoch 4/10
1563/1563  **19s** 12ms/step - accuracy: 0.6907 - loss: 0.8859 - val_accuracy: 0.6868 - val_loss: 0.9097

Epoch 5/10
1563/1563  **20s** 13ms/step - accuracy: 0.7169 - loss: 0.8068 - val_accuracy: 0.7076 - val_loss: 0.8606

Epoch 6/10
1563/1563  **21s** 13ms/step - accuracy: 0.7380 - loss: 0.7489 - val_accuracy: 0.7017 - val_loss: 0.8588

Epoch 7/10
1563/1563  **20s** 13ms/step - accuracy: 0.7572 - loss: 0.6953 - val_accuracy: 0.6911 - val_loss: 0.8975

Epoch 8/10
1563/1563  **21s** 13ms/step - accuracy: 0.7700 - loss: 0.6542 - val_accuracy: 0.7155 - val_loss: 0.8392

Epoch 9/10
1563/1563  **20s** 13ms/step - accuracy: 0.7871 - loss: 0.6064 - val_accuracy: 0.7092 - val_loss: 0.8766

Epoch 10/10
1563/1563  **20s** 13ms/step - accuracy: 0.8028 - loss: 0.5655 - val_accuracy: 0.7138 - val_loss: 0.8901

313/313  **2s** 5ms/step - accuracy: 0.7138 - loss: 0.8901

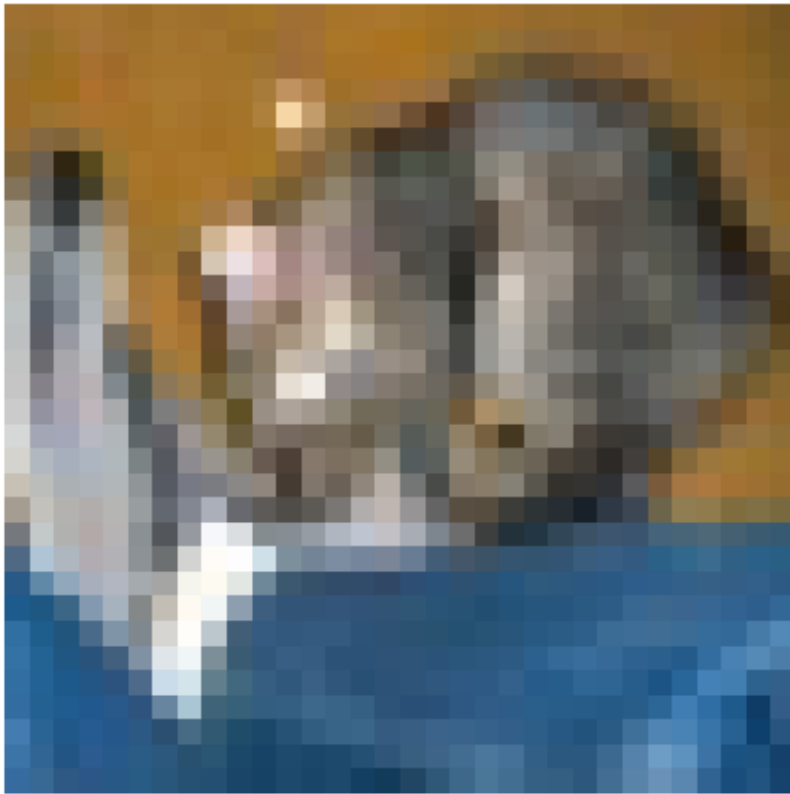
Test Accuracy: 0.7138000130653381

313/313  **1s** 4ms/step

Predicted Class: cat

Actual Class: cat

Predicted: cat



In []: